

Real Time Systems

(CT087-3-3-RTS)

Individual Assignment

Student Name : **Lim Beng Rhui**

TP Number : **TP068495**

Intake Code : **APD3F2601CS(DA)**

Lecturer Name : **Dr. Kuan Yik Junn**

Hand out Date : **2 March 2026**

Hand in Date : **9 May 2026**

Abstract

Computer systems in today's world shall have the capability to produce accurate results in a timely manner to allow decisions to be made in split seconds, thereby signifying the importance of real-time concepts in supporting the logical and temporal correctness of systems. Hence, this report designs a firm real-time system named `WikiRT` that retrieves event streams from Wikipedia's API to maintain a live leaderboard. Using the Rust programming language, `WikiRT` is built on two concurrency models that utilises threads and `Tokio` tasks respectively, with advanced techniques like zero-copy parsing implemented. Benchmark results from `Criterion.rs` show that thread-based model outperforms the asynchronous model by having a 20% higher throughput and 20% lower P99 latency, and using `DashMap` as a data structure to store leaderboard-related data is able to yield shorter execution time compared to traditional locking mechanisms.

Keywords. Real-time systems, Rust programming, concurrency models, zero-copy parsing, priority scheduling.

Table of Contents

Abstract 2
Introduction 4
Project Objective 5
Literature Review 6
Concepts in Real-Time Systems 6
Concurrency Models in Rust 7
Memory Management 8
Scheduling and Priority Queueing 9
System Design 10
Overall Architecture 10
Concurrency Models and Architecture 11
Thread-Based Model 11
Asynchronous Model 12
Inter-Process Communication (IPC) 13
Backpressure Management 13
Priority Scheduling 19
Memory Optimization 20
Shared Resource Synchronisation 22
Fault Tolerance and Resilience 23
Watchdog Timer 23
Fail-Safe and Degraded Mode 24
Results and Discussion 25
Pipeline Performance 25
Shared Resource Synchronisation and Lock Contention 27
Memory Allocation Performance 28
Advanced Features Integration 29
DashMap 29
Multi-tier fail-safe logic 30
Latency histogram 31
Fault injection 32
Conclusion 33
References 34

Introduction

In today's world, having systems that produce accurate results is no longer sufficient. Ranging from financial platforms that perform stock price prediction (Kalra et al., 2024) to scientific programs that involve heavy tasks like space mission monitoring (Watari, 2009), these systems not only demand for correct results, but the timeliness of these outputs also plays a critical role in ensuring that decisions can be made within seconds to avoid negative consequences like financial losses and safety risks from taking place. This gave birth to the concept of real-time systems that consider accuracy and timeliness into factors when determining the reliability of its outputs.

However, building a real-time system can be a challenging process, particularly with the use of traditional programming languages which come with their own trade-offs. Although high-level programming languages such as Java and Python offer programmers a high level of abstraction with plenty of libraries to assist in software development (Azenkouk, 2025), they also come with mechanisms like garbage collection that are non-deterministic in nature, affecting the performance stability of the systems developed with these languages (Abdulla, 2025). On the other hand, low-level languages like C allows programmers to directly control systems' memory and resource usage, but in the meantime also introduces memory safety issues and increased coding complexity as the size of the system expands (Yang, 2023).

To address these issues, in recent years, programming languages like as Rust has been invented to provide a combination of safety, convenience and performance in coding. First released in 2015, the Rust programming language implements several mechanisms, such as ownership model, borrowing rules, and compile-time memory safe guarantees, to provide high-level abstractions like Java while permitting low-level resource control like C without compromising on its safety and performance (Xu et al., 2021). This has made Rust stand out among the other programming languages to be one of the best choices when implementing real-time systems that require high performance while supporting concurrency.

Therefore, to investigate the internal mechanisms of Rust in supporting real-time systems, this report looks into the design and development of `WikiRT`, a real-time event processing system that receives real-time event streams from the Wikipedia API to update and display the website's top edited domains. Different techniques, such as priority queueing and zero-copy parsing, have been implemented to achieve high performance while minimising memory allocation within the system.

Project Objective

The objectives of this project include:

- To implement thread-based and asynchronous concurrency architectures to process Wikipedia event streams and update a leaderboard with top edited domains.
- To implement priority scheduling and zero-copy parsing techniques to reduce memory allocation and improve overall pipeline efficiency.
- To conduct performance benchmarking using `Criterion.rs` and measure throughput, latency, and parsing allocation efficiency across different implementation designs.

Literature Review

To gain a deeper understanding towards the different implementation designs available in real-time systems, a review towards existing works is performed based on several domains.

Concepts in Real-Time Systems

Some of the fundamental concepts of real-time systems to be discussed include:

- **Logical and temporal correctness**

  The correctness of real-time systems is measured on two aspects, namely (1) logical correctness, where the system shall produce outputs that are accurate and precise; and (2) temporal correctness, where the output must be delivered within a specific time constraint (Nkurikiyeyezu, 2022). Different types of testing methods, such as functional and structural testing, are well-established to measure a system's logical correctness (Khan, 2025). However, O'Sullivan et al. (1998) argues that performing testing on the system's temporal correctness often involves additional cost and complexity, where metrics like the worst-case execution time (Wang, 2017), as well as query and processing latency (Lokeswara et al., 2025), shall often be looked into to ensure the timeliness of outputs. In the `WikiRT` system, due to the high arrival speed of event streams, these temporal measurements are given higher importance to make sure that the top-edited domain leaderboard is updated within a fixed time interval.

- **Firm real-time systems**

  Real-time systems are classified into three based on deadline strictness, namely hard, firm and soft real-time systems. According to DataFlair (2021), hard real-time systems like aircraft systems handle deadlines strictly, where a miss in deadline is often considered system failure, while soft real-time systems like livestreaming platforms, on the other hand, is more lenient in handling late outputs as these outputs are still useful despite impacting user experience. Between these two categories lie the firm real-time system that aligns with `WikiRT`, where occasional deadline misses are allowed but the usefulness of these late results often degrades drastically over time (Al-Zaidy, 2023). To preserve the analytical values of the trends in Wikipedia edit, it is necessary for the `WikiRT` system to implement mechanisms like discarding events that exceeds a specific queueing delay threshold, so that resources will not be used to process stale data that offers less insights.

Concurrency Models in Rust

In Rust, two types of concurrency models can be implemented, which are:

- **Thread-based concurrency**

  The `WikiRT` system has implemented a thread-based design to allow multiple processes like data ingestion, processing and watchdog to run simultaneously. In Rust, threads are implemented using a 1:1 model, where creating a thread in Rust will create an operating-system level thread with its own stack (Klabnik et al., 2025). Although performance gain can be obtained by allowing these independent processes to run concurrently, thread-based models not only introduce higher coding complexity and potential issues like deadlocks and race conditions (MIT, n.d.) but are also costly when implemented in large scales due to the high memory usage and context-switching overhead required to execute and communicate between threads.

- **Asynchronous task-based concurrency**

  To tackle the drawbacks encountered by thread-based models, the Rust programming language has introduced an asynchronous task-based model that allows multiple tasks to be scheduled effectively within a minimal number of threads to support resource utilisation and code scalability (Flitton & Morton, 2024). Implemented using runtimes like `Tokio`, the asynchronous model uses cooperative multitasking to allow tasks to switch between one another when awaiting for other operations to complete (Medium, 2025a), which is particularly useful in the context of `WikiRT` as operations like watchdog monitoring can still be executed when no new events are received from Wikipedia Server-Sent Event (SSE) streams, thereby avoiding the system from requiring additional CPU threads that remain idle when waiting for I/O operations. However, unlike thread-based models that allows direct execution of tasks, the asynchronous model not only involves a scheduling overhead that may impact the program's overall performance but also introduces unnecessary complexity for simple and low concurrency tasks (Moya, 2025). Therefore, understanding project requirements and selecting the most appropriate concurrency model to be implemented is essential to yield optimal performance and resource utilisation.

Memory Management

Some of the aspects that are often explored to reduce memory consumption include:

- **Memory allocation in heap and stack**

  In Rust, heaps and stacks are two of the commonly used memory regions to allocate data, where each has its own strengths and drawbacks (Programiz, n.d.). With simple stack pointer adjustments, stacks are able to efficiently allocate and deallocate data following a last-in-first-out (LIFO) order, unlike heaps that require additional steps like bookkeeping to keep track of used memory and lead to performance overhead (Mathur, 2025). However, stack faces several limitations, such as having limited size and potential issues like stack overflow (Vasdeva, 2023), making it not suitable to be used in the `WikiRT` system when handling dynamically sized data from Wikipedia's SSE stream. Hence, to minimize memory copying events caused by heap reallocation operations, the `WikiRT` system shall pre-allocate a buffer to handle incoming events, so that the allocated memory is reused throughout the data streaming process and improve memory efficiency.

- **Zero-copy parsing**

  To further reduce memory allocation when handling input buffers, programmers have implemented the concept of zero-copy parsing in their systems, where programs create views or borrow references to the data instead of creating owned copies that occupies memory (Medium, 2025b). Although zero-copy parsing is commonly implemented in traditional languages like C and C++, these languages often introduce memory safety and require programmers to carefully handle memory management so that issues like dangling pointers will not take place (Vallabhaneni, 2025). Rust, on the other hand, offers different libraries like `nom`, `serde`, and `simd-json` to support zero-copy parsing while providing guarantee on memory safety with built-in mechanisms like borrowing and lifetimes (Stuart, 2023), not only allowing developers to be able to immediately catch memory-related errors at compile time to avoid spending additional time debugging for runtime issues, but also reducing heap allocations during the data ingestion phase to minimise memory consumption and processing latency. Looking at its importance, zero-copy parsing techniques are implemented in `WikiRT` to handle event streams from Wikipedia's API.

Scheduling and Priority Queueing

To efficiently control data flow, different scheduling and queueing mechanisms are explored:

- **Backpressure management**

  When a real-time system experiences high workload, having a backpressure management mechanism is essential to make sure that producer tasks do not overwhelm consumer tasks with excessive data (Phelps, 2019). The Rust programming language provides numerous abstractions to support backpressure management, where one of its most widely used approaches is by using bounded multi-producer single-consumer (MPSC) channels (Ghosh, 2026). The MPSC channel is incorporated into the `WikiRT` system, where a fixed capacity is specified so that during event spikes, producers will not continuously push events into the channel while providing consumers sufficient time to process events, thereby enhancing the responsiveness and stability of the overall ingestion and processing pipelines (Dagster, n.d.).

- **Priority-based scheduling**

  To ensure that computational resources are allocated efficiently, it is necessary to incorporate priority-based scheduling mechanisms to prevent systems from prioritising tasks with low importance and deliver results in an untimely manner (Brahmbhatt, 2025). Among the different priority scheduling techniques, multi-level queue scheduling has an advantage of providing a structural separation between different types of tasks, thereby providing the flexibility in defining the priority scheduling policies (Vidyalaya, 2026) and making it suitable to be implemented in the `WikiRT` system to distinguish the priority between human-edit and bot-edit events. However, the multi-level queue scheduling technique faces the starvation problem (Educative, n.d.), where the `WikiRT` system overcomes this by integrating an additional quota-based mechanism that defines a fixed ratio between high and low priority tasks to be executed in turns, thereby ensuring that both human-edit and bot-edit events can be processed.

System Design

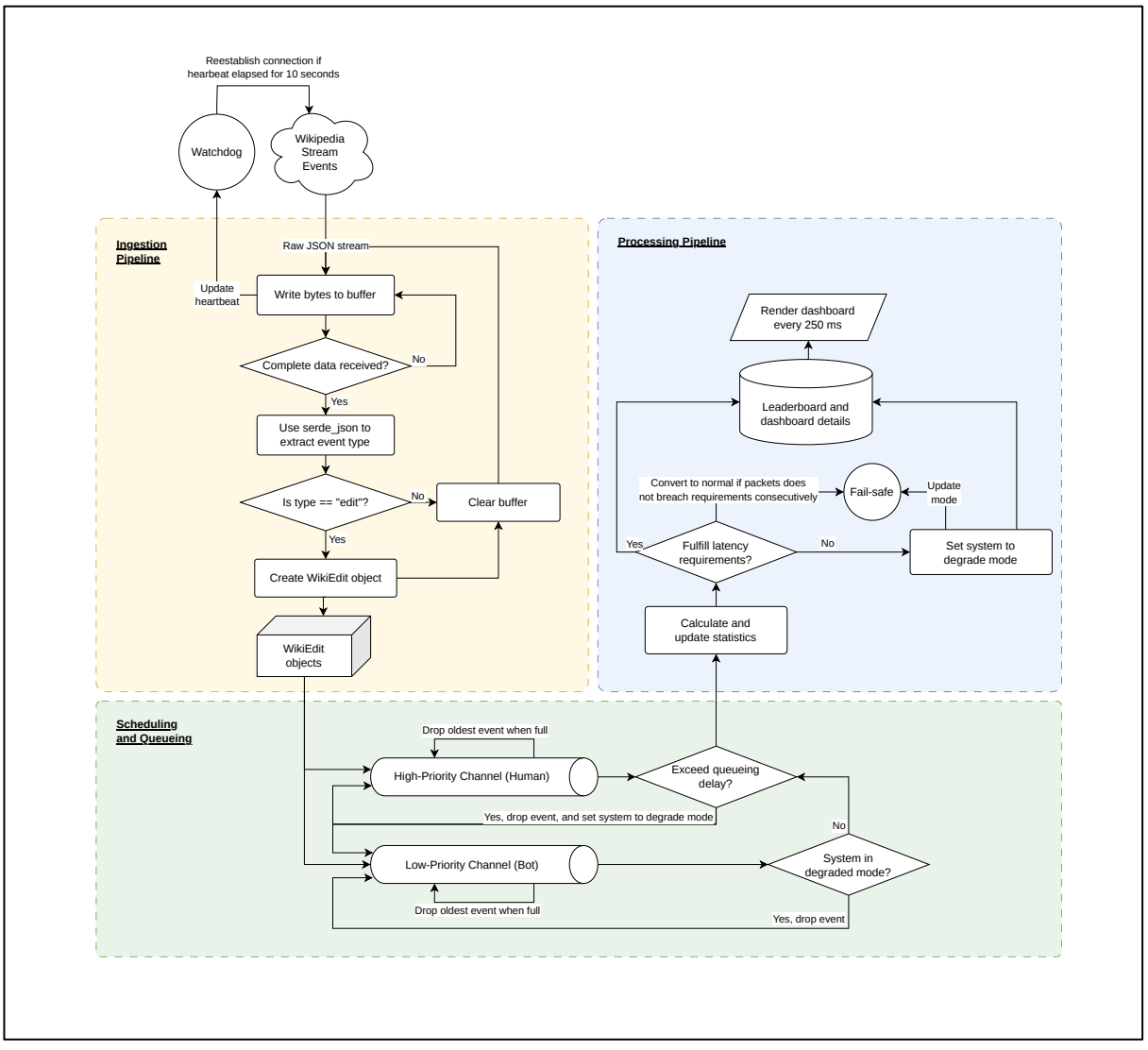
This section discusses the different components of the `WikiRT` system, together with the justification of these design choices.

Overall Architecture

In overall, the architectural design of the `WikiRT` system is shown in Figure 1, comprising different pipelines from data stream ingestion to event queueing and processing. Both the thread-based and asynchronous execution models follow a similar structural design to ensure fairness when evaluating their performances in subsequent phases.

Figure 1

Overall Architectural Design of `WikiRT`



Concurrency Models and Architecture

Two different concurrency models are implemented for the `WikiRT` project under the `src/pipeline/runner.rs` file.

Thread-Based Model

Threads in Rust are initialised using the `std::thread::spawn` function. As each thread in Rust corresponds to an operating system (OS)-level thread, using threads allow the system to achieve true parallelism, where each task can be executed simultaneously. However, OS-level threads often lead to higher overhead during thread creation and context switching. Although WikiRT only uses a few threads for its execution currently, the system's performance and efficiency will be significantly impacted when more threads are introduced to support additional features, making threads to be unsuitable when the system scales up.

```rust
pub fn run_threaded_pipeline(safety_handling: Safety, time_constraint: RunningTime) {

    // Initializing the variables to be used for threads here ...

    let allocator_dashboard = Arc::clone(&dashboard);
    let allocator_thread = thread::spawn(move || {
        run_allocator_monitor(shutdown_allocator, allocator_dashboard);
    });

    let mut ingest_thread = {

        // Clone items for the ingestion thread here ...

        thread::spawn(move || {
            let _ = sse_threaded::run(queue, hb, stop, &system_status);
        })
    };

    let processing_dashboard = Arc::clone(&dashboard);
    let processing_thread = thread::spawn(move || {

        // The operations for processing thread falls here ...
    });

    // Main thread
    loop {

        // Check for shutdown
        if start.elapsed() >= execute_duration {

            // Convert the flags to true
            shutdown_flag.store(true, Ordering::Relaxed);
            ingest_stop.store(true, Ordering::Relaxed);

            // Let all the threads end
            let _ = ingest_thread.join();
            let _ = processing_thread.join();
            let _ = monitoring_thread.join();
            let _ = allocator_thread.join();

            // Once all threads end, can exit the pipeline
            break;
        }

        // Check for watchdog by looking at if the watchdog channel sent anything
        if reset_rx.try_recv().is_ok() {

            // Perform operations to restart the ingest thread here ...
        }
    }
}
```

Asynchronous Model

The `Tokio` runtime is used to implement the asynchronous architecture of `WikiRT`, where tasks are distributed across a pool of worker threads via cooperative scheduling, allowing them to execute without occupying additional CPU threads. This makes the asynchronous approach suitable to be implemented in systems like `WikiRT` as it not only improves resource utilisation during tasks' idle periods, but also allows the system to expand and support large-scale operations. However, to coordinate between tasks, the asynchronous model also performs internal operations, making the model to introduce scheduling overhead and impact `WikiRT`'s performance, especially when only a small number of tasks is involved in the system.

```rust
pub async fn run_async_pipeline(safety_handling: Safety, time_constraint: RunningTime) -> Result<(), Box<dyn Error>> {

    // Initialize all data required for tasks here ...

    let watchdog_task = {
        // Clone the required data here ...
        tokio::spawn(async move {
            // Perform actions for watchdog here ...
        })
    };

    let allocation_task = {
        // Copy the required data here ...
        tokio::spawn(async move {
            run_allocator_monitor(shutdown, dashboard);
        })
    };

    let mut ingest_handle = {
        // Copy the required data here ...
        tokio::spawn(async move {
            let _ = sse_async::run(queue, heartbeat, ingest_stop, &system_state).await;
        })
    };

    let processing_task = {
        // Clone the required data here ...
        tokio::spawn(async move {
            // Perform actions for processing here ...
        })
    };

    // Main task
    loop {
        tokio::select! {

            _ = notify_ingest_restart.notified() => {
                // Perform task to stop and re-trigger ingestion
            }

            _ = &mut shutdown_timer => {
                // Perform actions to end the system
            }
        }
    }

    // Wait for all tasks to complete
    let _ = ingest_handle.await;
    let _ = processing_task.await;
    let _ = allocation_task.await;
    let _ = watchdog_task.await;

    // All tasks ended and can exit pipeline
    println!("Async pipeline stopped cleanly. Thanks for using the system!");
    Ok(())
}
```

Inter-Process Communication (IPC)

To allow different threads or tasks to perform tasks accurately while coordinating with one another, the message-passing model is implemented using MPSC channels under the Rust files in `src/pipeline`.

Backpressure Management

To support backpressure management, the priority queues implemented in `WikiRT` consist of two bounded MPSC channels with a fixed capacity that prevent producers from continuously enqueue events into the channels when consumers are unable to process these events at the same rate. This prevents the system from experiencing unstable performances due to the uncontrolled memory growth of these queues. To further enforce backpressure management in `WikiRT`, a drop-oldest mechanism is also implemented to remove the oldest packet when the channels are full, which ensures that the most recent events are able to be processed to align with the objectives of firm real-time systems that prioritises output timeliness over completion.

```
impl PriorityQueueThread {

    pub fn new(capacity: usize, quota: usize, dashboard: Arc<Dashboard>) -> Arc<Self> {

        let (high_tx, high_rx) = std::sync::mpsc::sync_channel::<WikiEdit>(capacity);
        let (low_tx, low_rx) = std::sync::mpsc::sync_channel::<WikiEdit>(capacity);

        Arc::new(Self {
            high_tx,
            high_rx: Arc::new(Mutex::new(high_rx)),
            low_tx,
            low_rx: Arc::new(Mutex::new(low_rx)),
            high_quota: Mutex::new(quota),
            max_quota: quota,
            dashboard,
        })
    }

    fn send_with_drop(
        &self,
        tx: &SyncSender<WikiEdit>,
        rx: &Arc<Mutex<Receiver<WikiEdit>>>,
        event: WikiEdit,
        queue_name: &'static str,
    ) {

        match tx.try_send(event) {

            Ok(_) => {}
            Err(TrySendError::Full(event)) => {

                let dropped_bytes = rx.lock().unwrap().try_recv().ok()
                    .map(|ev| ev.memory_bytes_estimate())
                    .unwrap_or(0);
                let incoming_bytes = event.memory_bytes_estimate();
                let _ = tx.try_send(event);

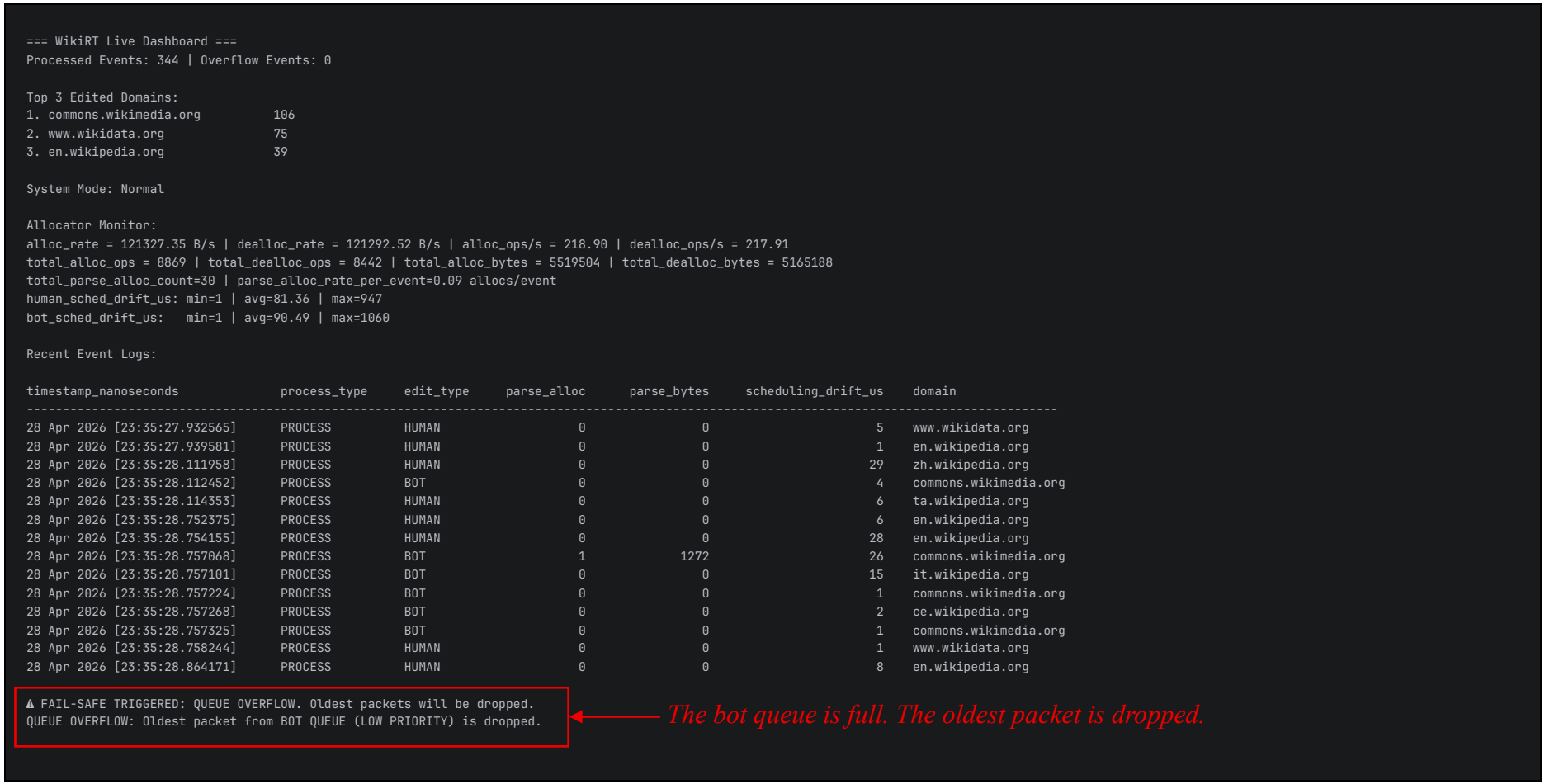
                self.dashboard.record_overflow_event(queue_name, dropped_bytes, incoming_bytes);
            }

            Err(TrySendError::Disconnected(_)) => {
                println!("Error: Cannot connect to channel");
            }
        }
    }
}
```

Figure 2 demonstrates backpressure management in `WikiRT` in action.

Figure 2

Backpressure Management in Action



```
=== WikiRT Live Dashboard ===
Processed Events: 344 | Overflow Events: 0

Top 3 Edited Domains:
1. commons.wikimedia.org          106
2. www.wikidata.org               75
3. en.wikipedia.org               39

System Mode: Normal

Allocator Monitor:
alloc_rate = 121327.35 B/s | dealloc_rate = 121292.52 B/s | alloc_ops/s = 218.90 | dealloc_ops/s = 217.91
total_alloc_ops = 8869 | total_dealloc_ops = 8442 | total_alloc_bytes = 5519504 | total_dealloc_bytes = 5165188
total_parse_alloc_count=30 | parse_alloc_rate_per_event=0.09 allocs/event
human_sched_drift_us: min=1 | avg=81.36 | max=947
bot_sched_drift_us:   min=1 | avg=90.49 | max=1060

Recent Event Logs:

timestamp_nanoseconds              process_type     edit_type     parse_alloc      parse_bytes     scheduling_drift_us    domain
-------------------------------------------------------------------------------------------------------------------------------------
28 Apr 2026 [23:35:27.932565]      PROCESS          HUMAN                   0                0                       5    www.wikidata.org
28 Apr 2026 [23:35:27.939581]      PROCESS          HUMAN                   0                0                       1    en.wikipedia.org
28 Apr 2026 [23:35:28.111958]      PROCESS          HUMAN                   0                0                      29    zh.wikipedia.org
28 Apr 2026 [23:35:28.112452]      PROCESS          BOT                     0                0                       4    commons.wikimedia.org
28 Apr 2026 [23:35:28.114353]      PROCESS          HUMAN                   0                0                       6    ta.wikipedia.org
28 Apr 2026 [23:35:28.752375]      PROCESS          HUMAN                   0                0                       6    en.wikipedia.org
28 Apr 2026 [23:35:28.754155]      PROCESS          HUMAN                   0                0                      28    en.wikipedia.org
28 Apr 2026 [23:35:28.757068]      PROCESS          BOT                     1             1272                      26    commons.wikimedia.org
28 Apr 2026 [23:35:28.757101]      PROCESS          BOT                     0                0                      15    it.wikipedia.org
28 Apr 2026 [23:35:28.757224]      PROCESS          BOT                     0                0                       1    commons.wikimedia.org
28 Apr 2026 [23:35:28.757268]      PROCESS          BOT                     0                0                       2    ce.wikipedia.org
28 Apr 2026 [23:35:28.757325]      PROCESS          BOT                     0                0                       1    commons.wikimedia.org
28 Apr 2026 [23:35:28.758244]      PROCESS          HUMAN                   0                0                       1    www.wikidata.org
28 Apr 2026 [23:35:28.864171]      PROCESS          HUMAN                   0                0                       8    en.wikipedia.org

⚠ FAIL-SAFE TRIGGERED: QUEUE OVERFLOW. Oldest packets will be dropped.
QUEUE OVERFLOW: Oldest packet from BOT QUEUE (LOW PRIORITY) is dropped.
```

Figure 2 Continued

```
=== WikiRT Live Dashboard ===
Processed Events: 346 | Overflow Events: 1

Top 3 Edited Domains:
1. commons.wikimedia.org          107
2. www.wikidata.org               76
3. en.wikipedia.org               39

System Mode: Normal

Allocator Monitor:
alloc_rate = 121327.35 B/s | dealloc_rate = 121292.52 B/s | alloc_ops/s = 218.90 | dealloc_ops/s = 217.91
total_alloc_ops = 8869 | total_dealloc_ops = 8442 | total_alloc_bytes = 5519504 | total_dealloc_bytes = 5165188
total_parse_alloc_count=30 | parse_alloc_rate_per_event=0.09 allocs/event
human_sched_drift_us: min=1 | avg=80.55 | max=947
bot_sched_drift_us:   min=1 | avg=90.49 | max=1060

Recent Event Logs:

timestamp_nanoseconds             process_type     edit_type     parse_alloc      parse_bytes     scheduling_drift_us    domain
28 Apr 2026 [23:35:28.112452]     PROCESS          BOT                     0                0                      4    commons.wikimedia.org
28 Apr 2026 [23:35:28.114353]     PROCESS          HUMAN                   0                0                      6    ta.wikipedia.org
28 Apr 2026 [23:35:28.752375]     PROCESS          HUMAN                   0                0                      6    en.wikipedia.org
28 Apr 2026 [23:35:28.754155]     PROCESS          HUMAN                   0                0                     28    en.wikipedia.org
28 Apr 2026 [23:35:28.757068]     PROCESS          BOT                     1             1272                     26    commons.wikimedia.org
28 Apr 2026 [23:35:28.757101]     PROCESS          BOT                     0                0                     15    it.wikipedia.org
28 Apr 2026 [23:35:28.757224]     PROCESS          BOT                     0                0                      1    commons.wikimedia.org
28 Apr 2026 [23:35:28.757268]     PROCESS          BOT                     0                0                      2    ce.wikipedia.org
28 Apr 2026 [23:35:28.757325]     PROCESS          BOT                     0                0                      1    commons.wikimedia.org
28 Apr 2026 [23:35:28.758244]     PROCESS          HUMAN                   0                0                      1    www.wikidata.org
28 Apr 2026 [23:35:28.864171]     PROCESS          HUMAN                   0                0                      8    en.wikipedia.org
28 Apr 2026 [23:35:29.167460]     PROCESS          HUMAN                   0                0                      9    commons.wikimedia.org
28 Apr 2026 [23:35:29.168795]     PROCESS          HUMAN                   0                0                      2    www.wikidata.org
28 Apr 2026 [23:35:29.577077]     OVERFLOW         -                       0             1290                      -    BOT QUEUE (LOW PRIORITY) dropped_oldest=1.6KB incoming=1.3KB
```

The overflow event is updated.

The dropped packet is logged in the table.

Figure 2 Continued

```
=== WikiRT Live Dashboard ===
Processed Events: 347 | Overflow Events: 1

Top 3 Edited Domains:
1. commons.wikimedia.org          107
2. www.wikidata.org               76
3. en.wikipedia.org               39

System Mode: Degraded
```

The system mode is changed to degraded mode.

```
Allocator Monitor:
alloc_rate = 121327.35 B/s | dealloc_rate = 121292.52 B/s | alloc_ops/s = 218.90 | dealloc_ops/s = 217.91
total_alloc_ops = 8869 | total_dealloc_ops = 8442 | total_alloc_bytes = 5519504 | total_dealloc_bytes = 5165188
total_parse_alloc_count=30 | parse_alloc_rate_per_event=0.09 allocs/event
human_sched_drift_us: min=1 | avg=80.55 | max=947
bot_sched_drift_us:   min=1 | avg=95.45 | max=1060

Recent Event Logs:

timestamp_nanoseconds             process_type     edit_type     parse_alloc      parse_bytes     scheduling_drift_us    domain
-------------------------------------------------------------------------------------------------------------------------------------------
28 Apr 2026 [23:35:28.114353]      PROCESS          HUMAN                   0                0                       6    ta.wikipedia.org
28 Apr 2026 [23:35:28.752375]      PROCESS          HUMAN                   0                0                       6    en.wikipedia.org
28 Apr 2026 [23:35:28.754155]      PROCESS          HUMAN                   0                0                      28    en.wikipedia.org
28 Apr 2026 [23:35:28.757068]      PROCESS          BOT                     1             1272                      26    commons.wikimedia.org
28 Apr 2026 [23:35:28.757101]      PROCESS          BOT                     0                0                      15    it.wikipedia.org
28 Apr 2026 [23:35:28.757224]      PROCESS          BOT                     0                0                       1    commons.wikimedia.org
28 Apr 2026 [23:35:28.757268]      PROCESS          BOT                     0                0                       2    ce.wikipedia.org
28 Apr 2026 [23:35:28.757325]      PROCESS          BOT                     0                0                       1    commons.wikimedia.org
28 Apr 2026 [23:35:28.758244]      PROCESS          HUMAN                   0                0                       1    www.wikidata.org
28 Apr 2026 [23:35:28.864171]      PROCESS          HUMAN                   0                0                       8    en.wikipedia.org
28 Apr 2026 [23:35:29.167460]      PROCESS          HUMAN                   0                0                       9    commons.wikimedia.org
28 Apr 2026 [23:35:29.168795]      PROCESS          HUMAN                   0                0                       2    www.wikidata.org
28 Apr 2026 [23:35:29.577077]      OVERFLOW         -                       0             1290                       -    BOT QUEUE (LOW PRIORITY) dropped_oldest=1.6KB incoming=1.3KB
28 Apr 2026 [23:35:29.577476]      PROCESS          BOT                     0                0                     880    ce.wikipedia.org
```

Figure 2 Continued

```
=== WikiRT Live Dashboard ===
Processed Events: 355 | Overflow Events: 1

Top 3 Edited Domains:
1. commons.wikimedia.org          109
2. www.wikidata.org               76
3. ce.wikipedia.org               40

System Mode: Degraded

Allocator Monitor:
alloc_rate = 179003.17 B/s | dealloc_rate = 178910.44 B/s | alloc_ops/s = 258.24 | dealloc_ops/s = 258.24
total_alloc_ops = 9177 | total_dealloc_ops = 8750 | total_alloc_bytes = 5702580 | total_dealloc_bytes = 5348171
total_parse_alloc_count=34 | parse_alloc_rate_per_event=0.10 allocs/event
human_sched_drift_us: min=1 | avg=79.62 | max=947
bot_sched_drift_us:   min=1 | avg=111.62 | max=1060

Recent Event Logs:

timestamp_nanoseconds             process_type     edit_type     parse_alloc      parse_bytes     scheduling_drift_us    domain
-------------------------------------------------------------------------------------------------------------------------------------------
28 Apr 2026 [23:35:28.758244]     PROCESS          HUMAN                   0                0                       1    www.wikidata.org
28 Apr 2026 [23:35:28.864171]     PROCESS          HUMAN                   0                0                       8    en.wikipedia.org
28 Apr 2026 [23:35:29.167460]     PROCESS          HUMAN                   0                0                       9    commons.wikimedia.org
28 Apr 2026 [23:35:29.168795]     PROCESS          HUMAN                   0                0                       2    www.wikidata.org
28 Apr 2026 [23:35:29.577077]     OVERFLOW         -                       0             1290                       -    BOT QUEUE (LOW PRIORITY) dropped_oldest=1.6KB incoming=1.3KB
28 Apr 2026 [23:35:29.577476]     PROCESS          BOT                     0                0                     880    ce.wikipedia.org
28 Apr 2026 [23:35:29.577688]     PROCESS          BOT                     0                0                     809    commons.wikimedia.org
28 Apr 2026 [23:35:29.577731]     PROCESS          BOT                     0                0                     749    commons.wikimedia.org
28 Apr 2026 [23:35:29.577764]     PROCESS          BOT                     0                0                     743    ce.wikipedia.org
28 Apr 2026 [23:35:29.577792]     PROCESS          BOT                     0                0                     716    ce.wikipedia.org
28 Apr 2026 [23:35:30.396144]     PROCESS          HUMAN                   3             1428                      13    pl.wikipedia.org
28 Apr 2026 [23:35:30.396459]     PROCESS          HUMAN                   0                0                     116    th.wikipedia.org
28 Apr 2026 [23:35:30.605146]     PROCESS          HUMAN                   0                0                       6    ja.wikipedia.org
28 Apr 2026 [23:35:30.907658]     PROCESS          HUMAN                   1             1045                       9    en.wikipedia.org

✅ FAIL-SAFE: DEGRADED → NORMAL. Bot edits will be resumed.
```

Once performance becomes stable, system mode reverts to normal.

Figure 2 Continued

```
=== WikiRT Live Dashboard ===
Processed Events: 356 | Overflow Events: 1

Top 3 Edited Domains:
1. commons.wikimedia.org          109
2. www.wikidata.org               76
3. ce.wikipedia.org               40

System Mode: Normal
```

System mode is updated.

```
Allocator Monitor:
alloc_rate = 179003.17 B/s | dealloc_rate = 178910.44 B/s | alloc_ops/s = 258.24 | dealloc_ops/s = 258.24
total_alloc_ops = 9177 | total_dealloc_ops = 8750 | total_alloc_bytes = 5702580 | total_dealloc_bytes = 5348171
total_parse_alloc_count=34 | parse_alloc_rate_per_event=0.10 allocs/event
human_sched_drift_us: min=1 | avg=80.71 | max=947
bot_sched_drift_us:   min=1 | avg=111.62 | max=1060

Recent Event Logs:

timestamp_nanoseconds             process_type     edit_type     parse_alloc      parse_bytes     scheduling_drift_us    domain
-------------------------------------------------------------------------------------------------------------------------------------------
28 Apr 2026 [23:35:28.864171]      PROCESS          HUMAN                   0                0                       8    en.wikipedia.org
28 Apr 2026 [23:35:29.167460]      PROCESS          HUMAN                   0                0                       9    commons.wikimedia.org
28 Apr 2026 [23:35:29.168795]      PROCESS          HUMAN                   0                0                       2    www.wikidata.org
28 Apr 2026 [23:35:29.577077]      OVERFLOW         -                       0             1290                       -    BOT QUEUE (LOW PRIORITY) dropped_oldest=1.6KB incoming=1.3KB
28 Apr 2026 [23:35:29.577476]      PROCESS          BOT                     0                0                     880    ce.wikipedia.org
28 Apr 2026 [23:35:29.577688]      PROCESS          BOT                     0                0                     809    commons.wikimedia.org
28 Apr 2026 [23:35:29.577731]      PROCESS          BOT                     0                0                     749    commons.wikimedia.org
28 Apr 2026 [23:35:29.577764]      PROCESS          BOT                     0                0                     743    ce.wikipedia.org
28 Apr 2026 [23:35:29.577792]      PROCESS          BOT                     0                0                     716    ce.wikipedia.org
28 Apr 2026 [23:35:30.396144]      PROCESS          HUMAN                   3             1428                      13    pl.wikipedia.org
28 Apr 2026 [23:35:30.396459]      PROCESS          HUMAN                   0                0                     116    th.wikipedia.org
28 Apr 2026 [23:35:30.605146]      PROCESS          HUMAN                   0                0                       6    ja.wikipedia.org
28 Apr 2026 [23:35:30.907658]      PROCESS          HUMAN                   1             1045                       9    en.wikipedia.org
28 Apr 2026 [23:35:30.907965]      PROCESS          HUMAN                   0                0                     289    he.wikipedia.org
```

Priority Scheduling

Priority scheduling is also implemented in `WikiRT` to allow the system to prioritise on processing human-edit events when facing resource constraints, as these events often reflect meaningful trends among Wikipedia edits. The scheduling strategy in `WikiRT` is adopted from multi-level queue scheduling, where human-edit and bot-edit events are separated into independent queues to allow programmers to define scheduling policies flexibly while having a structured pipeline.

To mitigate starvation issues that exist in traditional multi-level queue scheduling, a quota-based mechanism is introduced to ensure that bot-edit events can be processed once the quota for human-edit events is exhausted. As shown in the `recv()` function, whenever events are retrieved from the priority queue, the system will always check and process human-edit events first before switching to bot-edit events once the quota has reached, thereby ensuring that human-edit events always maintain a high priority while also enforcing scheduling fairness by making sure that bot-edit events will have a chance to be processed too.

```
pub struct PriorityQueueAsync {
    high_tx: Sender<WikiEdit>,
    high_rx: Arc<Mutex<Receiver<WikiEdit>>>,

    low_tx: Sender<WikiEdit>,
    low_rx: Arc<Mutex<Receiver<WikiEdit>>>,

    // Quota is used to let bot-edits run to prevent starvation
    high_quota: Arc<Mutex<usize>>,
    max_quota: usize,

    // And also linked to a dashboard to record any events taking place
    dashboard: Arc<Dashboard>,
}


impl PriorityQueueThread {

    /// A function to retrieve event / packet from channel
    pub fn recv(&self) -> Option<WikiEdit> {

        // We first lock the high_quota coz we'll update it
        let mut quota = self.high_quota.lock().unwrap();

        // Check for the quota. If still got quota we extract human-edit first
        if *quota > 0 && let Ok(event) = self.high_rx.lock().unwrap().try_recv() {
            *quota -= 1;
            return Some(event);
        }

        // If no more quota, we extract the bot-edits one time, then reset the quota
        if let Ok(event) = self.low_rx.lock().unwrap().try_recv() {
            *quota = self.max_quota;
            return Some(event);
        }

        // If no bot-edits and no more quota, then we continue extract human-edits and reset quota
        if let Ok(event) = self.high_rx.lock().unwrap().try_recv() {
            *quota = self.max_quota;
            return Some(event);
        }

        // If really have nothing to extract, then return none
        None
    }
}
```

Memory Optimization

To reduce memory allocation when handling input streams from Wikipedia's API, several mechanisms, such as lifetimes and zero-copy parsing techniques, are implemented in `WikiRT`. From the `run_async_pipeline()` function, the `process_event()` function is used to extract information from raw JSON data. By using the `serde_json` library, together with Rust lifetimes `<'a>`, the key fields are extracted and stored as `&'a str` without converting to self-owned `String` values (Serde, 2022). This allows the system to be able to reference existing data without involving any additional heap allocation to reduce memory copying, as evidenced by Figure 3 where the memory allocation count per event for the `process_event()` function is below 0.10, where the occasional spikes are caused by `serde`'s internal operations during the deserialization process, such as temporary buffer handling.

```rust
#[derive(Debug, Deserialize)]
pub struct WikiEvent<'a> {
    #[serde(default)]
    pub bot: Option<bool>,
    #[serde(default, borrow)]
    pub meta: Option<Meta<'a>>,
    #[serde(default, borrow, rename = "type")]
    pub change_type: Option<&'a str>,
}

#[derive(Debug, Deserialize)]
pub struct Meta<'a> {
    #[serde(default, borrow)]
    pub domain: Option<&'a str>,
}

pub fn process_event<'a>(raw_event: &'a [u8]) -> Result<Option<(WikiEvent<'a>, ParseMetrics)>, serde_json::Error> {

    let lines = raw_event.split(|&b| b == b'\n');
    for line in lines {
        // Check for data validity here ...

        // Use serde_json to extract the fields from the string
        let event = serde_json::from_slice::<WikiEvent<'a>>(data)?;

        // Return the WikiEvent object here ...
    }
    Ok(None)
}

pub async fn run_async_pipeline(safety_handling: Safety, time_constraint: RunningTime) -> Result<(), Box<dyn Error>> {

    // Other tasks in the async pipeline fall here ...

    let processing_task = {

        // Code for preparation falls here ...

        tokio::spawn(async move {
            loop {

                // Code to process shutdown operations falls here ...

                if let Some(event) = queue.recv().await {

                    // Perform checking on latency thresholds here ...

                    // Zero-copy parsing falls here
                    if let Ok(Some((wiki_edit, metrics))) = processor::process_event(&event.payload) {

                        // Update dashboard and record latency occurs here ...
    }; }) } } }

    // Remaining tasks to be executed here ...
}
```

Figure 3

Memory Allocation when Parsing JSON Data

```
=== WikiRT Live Dashboard ===
Processed Events: 1131 | Overflow Events: 0

Top 3 Edited Domains:
1. commons.wikimedia.org          338
2. www.wikidata.org               159
3. ce.wikipedia.org               154

System Mode: Normal

Allocator Monitor:
alloc_rate = 189850.40 B/s | dealloc_rate = 421398.91 B/s | alloc_ops/s = 439.79 | dealloc_ops/s = 572.12
total_alloc_ops = 43479 | total_dealloc_ops = 43109 | total_alloc_bytes = 18664988 | total_dealloc_bytes = 18531538
total_parse_alloc_count=72 | parse_alloc_rate_per_event=0.06 allocs/event
human_sched_drift_us: min=0 | avg=61.85 | max=571
bot_sched_drift_us:   min=0 | avg=64.03 | max=602

Recent Event Logs:

timestamp_nanoseconds              process_type     edit_type     parse_alloc      parse_bytes     scheduling_drift_us    domain
-------------------------------------------------------------------------------------------------------------------------------------------
28 Apr 2026 [21:33:43.069536]      PROCESS          HUMAN                    0                0                       4    es.wikipedia.org
28 Apr 2026 [21:33:43.605214]      PROCESS          HUMAN                    0                0                       2    en.wikipedia.org
28 Apr 2026 [21:33:43.768906]      PROCESS          BOT                      0                0                       3    commons.wikimedia.org
28 Apr 2026 [21:33:43.867291]      PROCESS          HUMAN                    0                0                      12    tr.wikipedia.org
28 Apr 2026 [21:33:43.868362]      PROCESS          BOT                      0                0                       0    sw.wikipedia.org
28 Apr 2026 [21:33:43.872138]      PROCESS          BOT                      0                0                       1    ce.wikipedia.org
28 Apr 2026 [21:33:44.096627]      PROCESS          BOT                      0                0                       2    commons.wikimedia.org
28 Apr 2026 [21:33:44.098549]      PROCESS          BOT                      0                0                       8    en.wikipedia.org
28 Apr 2026 [21:33:44.102269]      PROCESS          BOT                      0                0                       9    commons.wikimedia.org
28 Apr 2026 [21:33:44.103265]      PROCESS          BOT                      0                0                       8    ce.wikipedia.org
28 Apr 2026 [21:33:44.104018]      PROCESS          HUMAN                    0                0                       6    es.wikipedia.org
28 Apr 2026 [21:33:44.421480]      PROCESS          BOT                      0                0                       7    commons.wikimedia.org
28 Apr 2026 [21:33:44.421772]      PROCESS          BOT                      0                0                     293    commons.wikimedia.org
28 Apr 2026 [21:33:44.421777]      PROCESS          BOT                      0                0                     221    ce.wikipedia.org

Async pipeline stopped cleanly. Thanks for using the system!
```

```
=== WikiRT Live Dashboard ===
Processed Events: 1378 | Overflow Events: 0

Top 3 Edited Domains:
1. commons.wikimedia.org          418
2. www.wikidata.org               285
3. en.wikipedia.org               179

System Mode: Normal

Allocator Monitor:
alloc_rate = 214481.90 B/s | dealloc_rate = 225051.00 B/s | alloc_ops/s = 348.98 | dealloc_ops/s = 363.94
total_alloc_ops = 32958 | total_dealloc_ops = 32514 | total_alloc_bytes = 19031410 | total_dealloc_bytes = 18686097
total_parse_alloc_count=116 | parse_alloc_rate_per_event=0.08 allocs/event
human_sched_drift_us: min=0 | avg=82.48 | max=791
bot_sched_drift_us:   min=0 | avg=96.34 | max=821

Recent Event Logs:

timestamp_nanoseconds              process_type     edit_type     parse_alloc      parse_bytes     scheduling_drift_us    domain
-------------------------------------------------------------------------------------------------------------------------------------------
28 Apr 2026 [21:37:24.585025]      PROCESS          BOT                      0                0                     281    en.wikipedia.org
28 Apr 2026 [21:37:24.585029]      PROCESS          BOT                      0                0                     247    ce.wikipedia.org
28 Apr 2026 [21:37:24.585035]      PROCESS          BOT                      1             1990                       0    commons.wikimedia.org
28 Apr 2026 [21:37:24.585038]      PROCESS          HUMAN                    0                0                       0    www.wikidata.org
28 Apr 2026 [21:37:24.585101]      PROCESS          BOT                      0                0                      25    commons.wikimedia.org
28 Apr 2026 [21:37:24.585565]      PROCESS          HUMAN                    0                0                      23    ko.wikipedia.org
28 Apr 2026 [21:37:24.892669]      PROCESS          BOT                      0                0                      56    fr.wikipedia.org
28 Apr 2026 [21:37:24.892916]      PROCESS          HUMAN                    0                0                     308    www.wikidata.org
28 Apr 2026 [21:37:24.892925]      PROCESS          HUMAN                    0                0                     315    commons.wikimedia.org
28 Apr 2026 [21:37:24.892930]      PROCESS          HUMAN                    0                0                     301    fi.wikipedia.org
28 Apr 2026 [21:37:24.892933]      PROCESS          HUMAN                    0                0                     302    www.wikidata.org
28 Apr 2026 [21:37:24.892937]      PROCESS          BOT                      0                0                     318    ce.wikipedia.org
28 Apr 2026 [21:37:24.892945]      PROCESS          BOT                      0                0                     322    www.wikidata.org
28 Apr 2026 [21:37:24.892949]      PROCESS          BOT                      0                0                     312    commons.wikimedia.org

Async pipeline stopped cleanly. Thanks for using the system!
```

Shared Resource Synchronisation

As the `WikiRT` system delivers a leaderboard that records the top edited Wikipedia domains, when performing updates towards the leaderboard, it is necessary to make sure that no racing condition occurs to ensure the correctness of the displayed value. Therefore, the project has implemented four synchronisation techniques, including `Mutex`, `RwLock`, `Atomic` types and `DashMap`, under the file `src/leaderboard.rs`, to ensure that values are accessed and updated safely when multiple workers access the leaderboard. Although traditional locking mechanisms like `Mutex` and `RwLock` provide strong safety and consistency guarantees, they often lock the entire shared object when workers perform edits, leading to a higher blocking overhead if compared to newer concurrent structures like `DashMap` that uses sharded locking to allow workers to perform localised updates simultaneously (Navar, 2024).

```rust
/// Leaderboard using Mutex
pub struct MutexLeaderboard {
    inner: Mutex<HashMap<Arc<str>, u64>>,
}

impl MutexLeaderboard {
    pub fn new() -> Self {
        Self {
            inner: Mutex::new(HashMap::new()),
        }
    }
}

pub struct RwLeaderboard {
    inner: RwLock<HashMap<Arc<str>, u64>>,
}


/// Leaderboard using RwLock
impl RwLeaderboard {
    pub fn new() -> Self {
        Self {
            inner: RwLock::new(HashMap::new()),
        }
    }
}


/// Leaderboard using RwLock + Atomic
pub struct AtomicLeaderboard {
    inner: RwLock<HashMap<Arc<str>, AtomicU64>>,
}

impl AtomicLeaderboard {
    pub fn new() -> Self {
        Self {
            inner: RwLock::new(HashMap::new()),
        }
    }
}


/// Leaderboard using DashMap
pub struct DashMapLeaderboard {
    inner: DashMap<Arc<str>, AtomicU64>,
}

impl DashMapLeaderboard {
    pub fn new() -> Self {
        Self {
            inner: DashMap::new(),
        }
    }
}
```

Fault Tolerance and Resilience

To ensure that the system is able to maintain its performance under high-contention scenarios, several fault tolerance mechanisms have been implemented.

Watchdog Timer

Since the system may occasionally experience network interruptions during its execution, a watchdog mechanism is implemented, where if the `WikiRT` system does not receive any event stream for 10 seconds, the ingestion pipeline will be terminated and restarted. This allows the system to restore without manual intervention, thereby ensuring its availability and stability.

```rust
/// Create a Heartbeat type to record the time of the last seen packet
pub type Heartbeat = Arc<Mutex<Instant>>;


/// Watchdog struct has two data: last seen time and a transmitter to communicate with main thread
pub struct Watchdog {
    heartbeat: Heartbeat,
    reset_tx: SyncSender<()>,
}


impl Watchdog {

    /// Initialize the watchdog
    pub fn new(heartbeat: Heartbeat, reset_tx: SyncSender<()>) -> Self {
        Self {
            heartbeat,
            reset_tx,
        }
    }

    /// Function to trigger watchdog for constant monitoring
    pub fn run(&self, stop_flag: Arc<AtomicBool>) {
        loop {

            // Check if the thread has to be stopped
            if stop_flag.load(Ordering::Relaxed) {
                break;
            }

            // Check every 0.5 second (to prevent high CPU usage, okay for 10-second timeout)
            thread::sleep(Duration::from_millis(500));

            // Check for elapsed time
            let elapsed = self.heartbeat.lock().unwrap().elapsed();

            // If it is more than 10 seconds
            if elapsed >= Duration::from_secs(10) {

                // Send a trigger to the MPSC channel at the run function
                match self.reset_tx.try_send(()) {

                    // Send a log when trigger is sent
                    Ok(_) => {
                        println!("Watchdog Triggered: Network Reset in progress");
                    }

                    // Also do nothing if the channel is full, means that reset will take place
                    Err(std::sync::mpsc::TrySendError::Full(_)) => {}

                    // Prompt error when the channel cannot be accessed
                    Err(std::sync::mpsc::TrySendError::Disconnected(_)) => {
                        println!("Failed to send reset signal");
                    }
                }
            }
        }
    }
}
```

Fail-Safe and Degraded Mode

Aside from the watchdog timer, the system also incorporates a fail-safe mechanism, where the system will enter a degraded mode when several violations are met, such as when the processing channels are full or when the rolling P99 latency exceeds a defined threshold. Under degraded mode, the system will drop all bot-edit events and focus on processing human-edit events to maximise the priority of these events. The system will only recover back to its normal mode once stable performance is detected across a consecutive number of events.

```
impl FailSafe {

    /// A function to update packet latency into system
    pub fn record_latency(&self, latency: Duration) {

        // Try to get and update the latency histogram, then calculate P99
        let p99 = {
            let mut hist = self.histogram.lock().unwrap();
            hist.record(latency);
            hist.p99()
        };

        // Perform evaluation to determine if system should enter degrade mode
        self.update_status(latency, p99);
    }

    /// A function to record dropped packets and activate degrade mode
    pub fn report_dropped_packet(&self) {
        let mut mode = self.mode.lock().unwrap();
        let mut stable = self.stable_counter.lock().unwrap();

        *mode = SystemMode::Degraded;
        *stable = 0;

        println!("⚠ FAIL-SAFE TRIGGERED: QUEUE DELAY EXCEEDED. System overloaded.");
    }

    /// A function to evaluate if the system shall enter degrade mode
    fn update_status(&self, last_latency: Duration, p99: u64) {
        let mut mode = self.mode.lock().unwrap();
        let mut stable = self.stable_counter.lock().unwrap();

        // Case 1: Latency exceeds the 2ms deadline
        if last_latency > PROCESSING_JITTER_LIMIT {
            *mode = SystemMode::Degraded;
            *stable = 0;
            println!("⚠ FAIL-SAFE TRIGGERED: HARD DEADLINE BREACH");
            return;
        }

        // Case 2: Latency exceeds the P99 tail (from packet perspective)
        if last_latency.as_micros() as u64 > p99 {
            *mode = SystemMode::Degraded;
            *stable = 0;
            println!("⚠ FAIL-SAFE TRIGGERED: PACKET P99 BREACH. Bot edits will be dropped.");
            return;
        }

        // Case 3 (Additional): If the current P99 exceeds the defined P99 in the system (from system perspective)
        if p99 > FAIL_SAFE_P99_OVERALL_THRESHOLD_MICRO {
            *mode = SystemMode::Degraded;
            *stable = 0;
            println!("⚠ FAIL-SAFE TRIGGERED: SYSTEM P99 BREACH. Bot edits will be dropped.");
            return;
        }

        // If no degradation is triggered, latency is normal, so check if should return back to normal
        *stable += 1;
        if *mode == SystemMode::Degraded && *stable >= self.required_stable_count {
            *mode = SystemMode::Normal;
            *stable = 0;
            println!("✅ FAIL-SAFE: RECOVERY → NORMAL. Bot edits will be resumed.");
        }
    }
}
```

Results and Discussion

The `Criterion.rs` library is also implemented to rigorously evaluate the performance of the `WikiRT` system from different perspectives.

Pipeline Performance

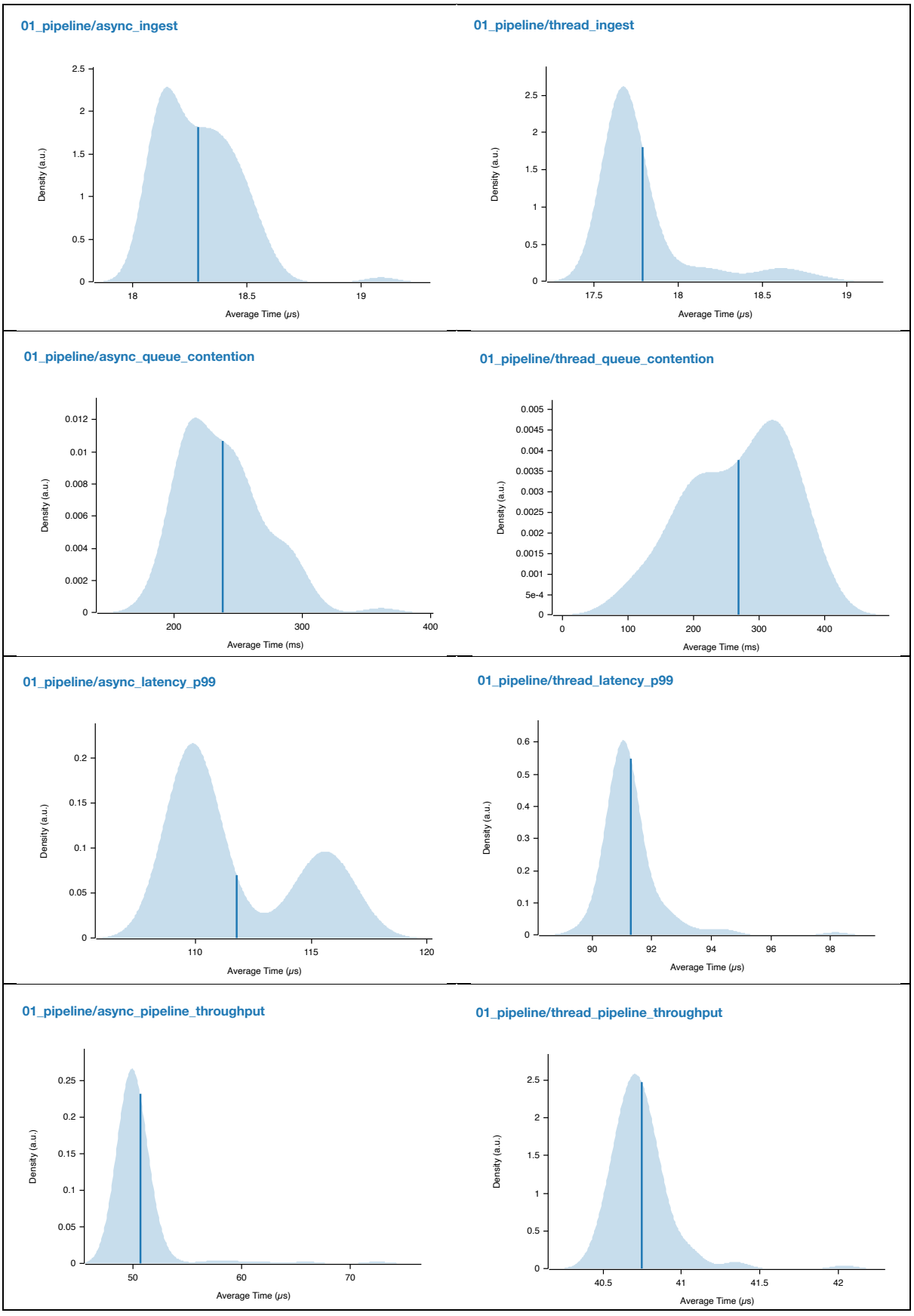
The thread-based and asynchronous concurrency models are compared against different processes, including data ingestion, queue contention, and the overall end-to-end pipeline. Based on the density plots in Figure 4, it is shown that:

- **Thread-based model shows better throughput and stability with lower tail latency**
  Comparing the average throughput and P99 tail latency of both models, the thread-based model is found to have an approximately 20% better performance, as well as a lower execution variance that indicates reduced jitter, compared to the asynchronous model. As both models are executed to perform repetitive tasks during benchmarking, this indicates that small number of threads, once properly established, will have a lower scheduling overhead and yield better performance, unlike Tokio tasks that rely on cooperative scheduling to coordinate among tasks, hence consistently introducing additional scheduling overhead that affects their overall performance. Furthermore, since threads in Rust are mapped to operating system (OS)-level threads, they are able to utilise hardware resources better, not only allowing them to perform different operations more efficiency but also contributing to a more deterministic behaviour when performing repetitive tasks.

- **Asynchronous model performs better under high concurrency**
  As the number of threads and tasks increase, the asynchronous model (237.69 ms) has a 10% faster execution time compared to the thread-based model (269.06 ms), with its standard deviation (31.995 ms) less than half of the thread-based model (77.243 ms). Such a scenario reflects the strength of the asynchronous model when handling a high number of concurrent tasks, where the cooperative scheduling technique implemented by Tokio is able to effectively create, suspend and resume thousands of tasks without overwhelming the CPU while providing a stable performance, despite experiencing subtle execution variability as shown by its bimodal distribution. On the other hand, the thread model will have to create a high number of threads that often leads to heavy overload when performing operations like thread creation and context switching, thereby imposing high pressure on the CPU and causes higher variation and degradation towards its performance when the workload scales up.

Figure 4

Performance Comparison between Thread-Based and Asynchronous Concurrency Models

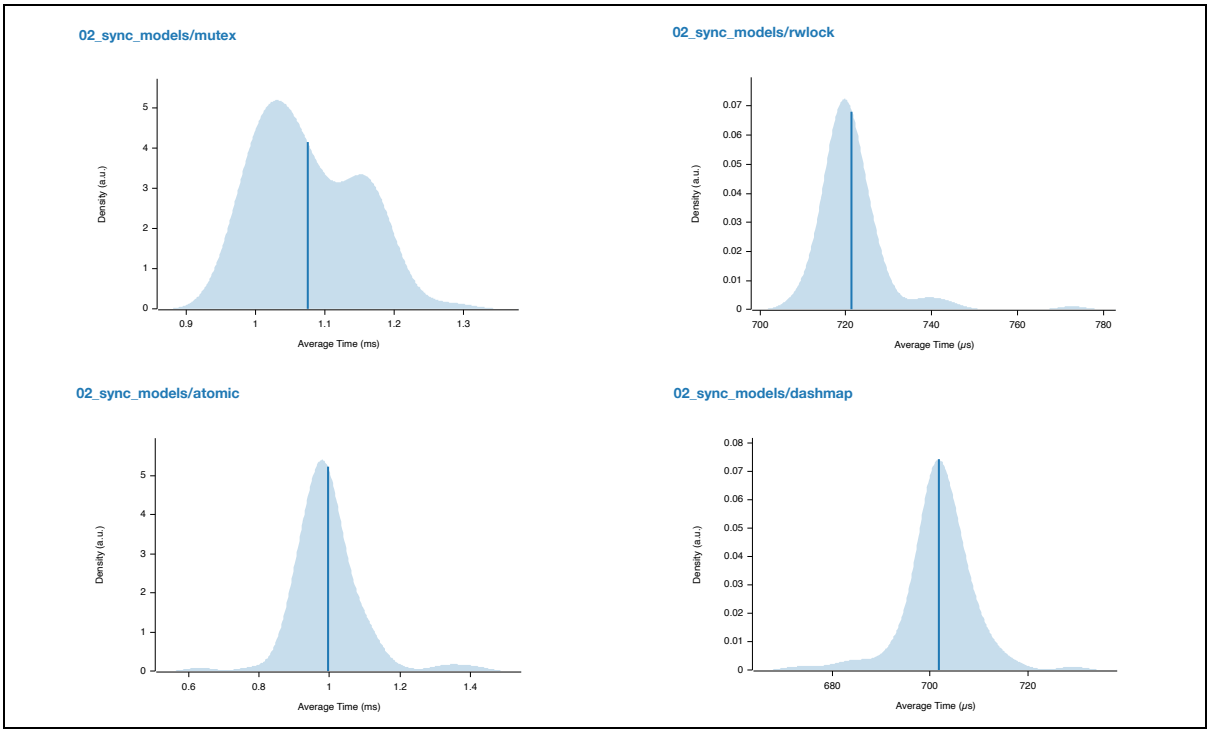


Shared Resource Synchronisation and Lock Contention

The performance of different locking mechanisms, including `Mutex`, `RwLock`, `Atomic` types, and `DashMap` are also evaluated under high contention, where 4 threads are spawned to individually execute 2000 update operations towards the leaderboard concurrently. From the results in Figure 5, it is revealed that `DashMap` has the shortest execution time of 701.85 µs, offering a similar performance as `RwLock` (721.45 µs) while outperforming other mechanisms like `Atomic` (995.05 µs) and `Mutex` (1.0753 ms). As `DashMap` implements a sharding locking mechanism that allows different tasks and threads to perform localised updates, this significantly improves `DashMap`'s throughput under high workloads, unlike mechanisms like `Mutex` that only allow one worker to access shared resources while blocking other workers, thereby increases lock contention and leads to a 50% longer execution time. Moreover, the smaller execution variance observed on `DashMap` and `RwLock` indicates that both mechanisms are able to provide a more stable and predictable performance when handling high concurrent workloads. However, considering the exclusive write locking mechanism of `RwLock` during data updates that slightly degrades its performance, `DashMap` stands out to be the best locking mechanism to be implemented in `WikiRT` due to its overall low execution time and stable performance that translates to higher throughput and better determinism during implementation.

Figure 5

Performance Comparison between Different Types of Shared Resource Synchronisation

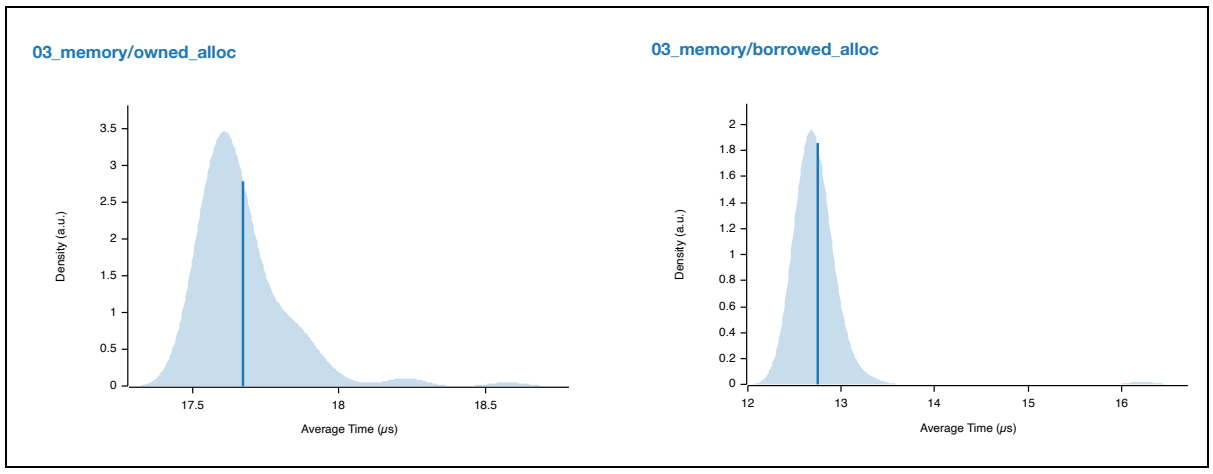


Memory Allocation Performance

To evaluate the effectiveness of zero-copy parsing in `WikiRT`, the benchmark has also measured the time taken for the system to deserialize JSON data into owned and borrowed data structures. Based on the result in Figure 6, the borrowed allocation approach achieves a mean execution time of 12.748 µs, which is approximately 25% faster than the owned allocation approach (17.675 µs). Such a scenario occurs as the borrowed allocation approach directly borrowed references from the original input without creating self-owned objects like `String`, hence minimising any additional operations like data copying and heap allocation that not only increases memory consumption but also reduces system throughput and performance. Although it is observed that the borrowed allocation approach has a slightly higher standard deviation (378.65 ns) compared with the owned allocation approach (163.52 ns), which is associated with the internal operations performed by `serde_json` during the deserialization process, the fluctuation that falls within nanoseconds is negligible when considering the microsecond-level improvements gained in higher execution time, making the borrowed allocation approach to be more appropriate to be implemented in real-time systems like `WikiRT` that treat lower latency and reduced dynamic memory allocations as critical aspects in maintaining temporal correctness.

Figure 6

Performance Comparison between Owned Allocation and Borrowed Allocation



Advanced Features Integration

Several additional features are also included in the `WikiRT` system, which include:

DashMap

Aside from traditional locking mechanisms, the `WikiRT` system has also provided the implementation of `DashMap` to update the counts for the edited domains in Wikipedia. The `DashMap` data structure is similar to `HashMap` but comes with additional concurrency features that automatically guarantee concurrency safety without requiring programmers to explicitly perform locking operations before performing any update tasks.

```rust
/// An additional comparison: DashMap that supports concurrency with atomic
pub struct DashMapLeaderboard {
    inner: DashMap<Arc<str>, AtomicU64>,
}


/// Constructor for DashMap-based leaderboard
impl DashMapLeaderboard {
    pub fn new() -> Self {
        Self {
            inner: DashMap::new(),
        }
    }
}


/// Functions to be overridden from parent
impl Leaderboard for DashMapLeaderboard {

    /// A function to update domains into the dashmap
    fn update(&self, domain: &str) {

        // First convert the domain string into arc
        let domain_arc = Arc::<str>::from(domain);

        // Directly insert values, don't have to lock (DashMap handles lock automatically)
        let counter = self
            .inner
            .entry(domain_arc)
            .or_insert_with(|| AtomicU64::new(0));
        counter.fetch_add(1, Ordering::Relaxed);
    }

    /// A function to return the top n domains
    fn top(&self, size: usize) -> Vec<(Arc<str>, u64)> {

        // Directly perform iteration - DashMap is safe
        let mut items: Vec<(Arc<str>, u64)> = self
            .inner
            .iter()
            .map(|entry| (entry.key().clone(), entry.value().load(Ordering::Relaxed)))
            .collect();

        // Sort the vector and truncate it to top n items
        items.sort_by(|a, b| b.1.cmp(&a.1));
        items.truncate(size);

        // Lastly return the vector list
        items
    }
}
```

Multi-tier fail-safe logic

Instead of relying solely on the static 2 milliseconds deadline, the `WikiRT` system consistently monitors the latency of each processing task and perform evaluation based on three different conditions, so that the system can enter `Degrade` mode and convert back to `Normal` mode dynamically, not only to maintain its stability but also to prioritise on human-edit events when experiencing high workload.

```rust
impl FailSafe {

/// A function to evaluate if the system shall enter degrade mode
fn update_status(&self, last_latency: Duration, p99: u64) {

    // First retrieve two information: system mode and stable count
    let mut mode = self.mode.lock().unwrap();
    let mut stable = self.stable_counter.lock().unwrap();

    // Perform checking on the latency
    // Case 1: Latency exceeds the 2ms deadline
    if last_latency > PROCESSING_JITTER_LIMIT {

        // Convert the system mode to degraded and reset stable to 0
        *mode = SystemMode::Degraded;
        *stable = 0;

        // Output and end the function
        println!("⚠ FAIL-SAFE TRIGGERED: HARD DEADLINE BREACH");
        return;
    }

    // Case 2: Latency exceeds the P99 tail (from packet perspective)
    if last_latency.as_micros() as u64 > p99 {

        // Also degrade the system and output the trigger
        *mode = SystemMode::Degraded;
        *stable = 0;
        println!("⚠ FAIL-SAFE TRIGGERED: PACKET P99 BREACH. Bot edits will be dropped.");
        return;
    }

    // Case 3 (Additional): If the current P99 exceeds the defined P99 in the system (from system perspective)
    if p99 > FAIL_SAFE_P99_OVERALL_THRESHOLD_MICRO {

        // Overall system performance is degrading, so degrade the system
        *mode = SystemMode::Degraded;
        *stable = 0;
        println!("⚠ FAIL-SAFE TRIGGERED: SYSTEM P99 BREACH. Bot edits will be dropped.");
        return;
    }

    // If no degradation is triggered, latency is normal, so stable can be incremented
    *stable += 1;

    // Check if the stable count has achieved the requirement
    if *mode == SystemMode::Degraded && *stable >= self.required_stable_count {

        // Reset the system to normal
        *mode = SystemMode::Normal;
        *stable = 0;

        // Output status
        println!("✅ FAIL-SAFE: DEGRADED → NORMAL. Bot edits will be resumed.");
    }
}
```

Latency histogram

To calculate the rolling 99th percentile of latency, instead of recording all records dynamically, the `WikiRT` system implements a latency histogram that categorises each measurement into predefined bins, where updates will only take place on the bin's frequency. Such an approach not only encourages constant-time updates to minimise processing latency, but also avoids unnecessary heap allocations, thereby contributing to the system's determinism.

```rust
pub(crate) struct LatencyHistogram {
    buckets: Vec<u64>,
    bucket_width: u64,
    total: u64,
    overflow: u64,
}

impl LatencyHistogram {

    /// A function to initialize itself
    pub(crate) fn new(total_range: usize, bucket_width: usize) -> Self {
        Self {
            buckets: vec![0; total_range / bucket_width],
            bucket_width: bucket_width as u64,
            total: 0,
            overflow: 0,
        }
    }

    /// A function to record the latency into the histogram
    pub(crate) fn record(&mut self, latency_in_micro: Duration) {

        // Calculate the position of the bin that the latency will fall under
        let bucket_position = (latency_in_micro.as_micros() as u64 / self.bucket_width) as usize;

        // If the latency does not belong to any bin, it goes to the overflow
        if bucket_position >= self.buckets.len() {
            self.overflow += 1;

        // If it's a normal value, it goes into the corresponding bin
        } else { self.buckets[bucket_position] += 1; }

        // Update the total count
        self.total += 1;
    }

    pub(crate) fn p99(&self) -> u64 {

        // Have to handle cold-start, if less than 50 records then use a predefined threshold
        if self.total < 50 {
            return FAIL_SAFE_P99_OVERALL_THRESHOLD_MICRO;
        }

        // Calculation: First find the position of P99
        let target = ((self.total as f64) * 0.99).ceil() as u64;

        // Then loop through each bucket from left to right to find the bucket
        let mut cumulative = 0;
        for (bucket_index, bucket_count) in self.buckets.iter().enumerate() {
            cumulative += *bucket_count;

            // Return the lower limit of the bucket
            if cumulative >= target {
                return (bucket_index as u64) * self.bucket_width;
            }
        }

        // If P99 is in the overflow, then return the max of the histogram
        cumulative += self.overflow;
        if cumulative >= target {
            return (self.buckets.len() as u64) * self.bucket_width;
        }

        // In case if calculation error occurs, use back the initial threshold
        FAIL_SAFE_P99_OVERALL_THRESHOLD_MICRO
    }
}
```

Fault injection

To ensure that the watchdog and fail-safe mechanisms in `WikiRT` are able to work as intended, synthetic fault injection is implemented into the system by using the rand library to generate a 0.5% possibility of letting processing tasks and threads to sleep for 500 milliseconds. This allows resource-constraint situations, such as queues reaching its maximum capacity, to be demonstrated so that the effectiveness of the safety mechanisms can be validated.

```rust
/// The function to execute async-based pipeline
pub async fn run_async_pipeline(safety_handling: Safety, time_constraint: RunningTime) -> Result<(), Box<dyn Error>> {

    // Resource and task initiation takes place here ...

    // Processing task
    let processing_task = {

        // Pointer cloning takes place here ...

        // Create the processing task
        tokio::spawn(async move {
            loop {

                // Handle shutdown actions here ...

                // When an event is received
                if let Some(event) = queue.recv().await {

                    // Fault injection: Randomly introduce sleeping in processing
                    if rand::random_range(1..=200) == 1 {
                        println!("FAULT INJECTION: Simulating 0.5s processing delay...");
                        tokio::time::sleep(Duration::from_millis(500)).await;
                    }

                    // Other actions to place event into queue takes place here ...
                }
            }
        })
    };

    // Other tasks continue here ...
}


/// The function to execute thread-based pipeline
pub fn run_threaded_pipeline(safety_handling: Safety, time_constraint: RunningTime) {

    // Resource and thread initiation takes place here ...

    // Processing thread
    let processing_thread = thread::spawn(move || {

        // Clone the dashboard to be updated later
        let dashboard = Arc::clone(&processing_dashboard);
        loop {

            // Handle shutdown events here ...

            // If the channel receives a packet
            if let Some(event) = process_queue.recv() {

                // Fault injection: Randomly introduce sleeping in processing
                if rand::random_range(1..=200) == 1 {
                    println!("FAULT INJECTION: Simulating 0.5s processing delay...");
                    thread::sleep(Duration::from_millis(500));
                }

                // Other operations to put events into queue take place here ...
            }
        }
    });

    // Other threads continue here ...
}
```

Conclusion

In summary, this report explores the different real-time system concepts that are incorporated into the `WikiRT` system. Designed with two concurrency architectures, the system utilises techniques like zero-copy parsing and backpressure management to process Wikipedia Server-Sent Events (SSE) stream and maintain a live leaderboard that shows the top edited Wikipedia domains. Aside from providing justification towards the different system design choices, `WikiRT`'s overall performance is also evaluated using the `Criterion.rs` library. The incorporation of additional features like `DashMap` and multi-tier fail-safe logic further enhances the system, allowing it to be able to fulfil the requirements of a firm real-time system with minimised memory allocation while maintaining logical and temporal correctness.

Despite the robust performance shown by the `WikiRT` system, several enhancements that can be performed include:

- introducing thread pools and task pools to allow multiple workers to process Wikipedia edits to yield higher throughput,
- integrating adaptive priority scheduling techniques to handle issues like starvation more effectively when the system performs under different workloads, and
- incorporating with third-party data analytics platforms like PowerBI to better visualise data and gain additional insights towards the editing trends in Wikipedia.

(4000 words)

References

Abdulla, M. (2025). VGC: A high-performance zone-based garbage collector architecture for Python with partitioning and parallel execution. *ArXiv, 2512.23768*, 1-30.

Al-Zaidy, H. A. (2023). Real Time System. 1-6.

Azenkouk, A. (2025). Python vs Java: Which Is better for beginners? *International Journal of Research and Innovation in Social Science (IJRISS), 9*(8), 2392-2397.

Brahmbhatt, R. (2025, October 29). *Operating system scheduling algorithms priority-based scheduling*. Retrieved from Medium: https://medium.com/@rudrab1914/operating-system-scheduling-algorithms-priority-based-scheduling-0d06b1ea51ed

Dagster. (n.d.). *Backpressure*. Retrieved from Dagster: https://dagster.io/glossary/data-backpressure

DataFlair. (2021, August 25). *Real Time Operating System*. Retrieved from DataFlair: https://data-flair.training/blogs/real-time-operating-system/

Educative. (n.d.). *What is multi-level queue scheduling?* Retrieved from Educative: https://www.educative.io/answers/what-is-multi-level-queue-scheduling

Flitton, M., & Morton, C. (2024). *Async Rust: Unleashing the Power of Fearless Concurrency.* O'Reilly Media.

Ghosh, D. (2026, February 9). *Backpressure in Async Rust: From pull-based streams to explicit sinks and semaphores*. Retrieved from LinkedIn: https://www.linkedin.com/pulse/backpressure-async-rust-from-pull-based-streams-explicit-ghosh-p8fyc

Kalra, R., Singh, T., Mishra, S., Satakshi, Kumar, N., Kim, T., & Kumar, M. (2024). An efficient hybrid approach for forecasting real-time stock market indices. *Journal of King Saud University - Computer and Information Sciences, 36*(8), 102180.

Khan, Y. (2025, November 1). *Unit testing vs integretion testing*. Retrieved from Medium: https://medium.com/@yasiffkhan/unit-testing-vs-integretion-testing-b6be58bddf10

Klabnik, S., Nichols, C., & Krycho, C. (2025). *Using Threads to run code simultaneously*. Retrieved from The Rust Programming Language: https://doc.rust-lang.org/book/ch16-01-threads.html

Lokeswara, A., Ilman, B., & Widyatama, N. (2025). Adaptive Kafka resource allocation for real-time monitoring in the manufacturing sector. *Research Square*, 1-23.

Mathur, S. (2025, August 13). *Stack vs heap in Rust: A complete memory management deep dive*. Retrieved from KodingKorp: https://www.kodingkorp.com/knowledge-base/stack-vs-heap-in-rust

Medium. (2025a, October 8). *Inside Rust's cooperative multitasking: The secret behind Tokio's fairness*. Retrieved from Medium: https://medium.com/@theopinionatedev/inside-rusts-cooperative-multitasking-the-secret-behind-tokio-s-fairness-a8caa2f79b81

Medium. (2025b, October 30). *Zero-copy or bust: Designing a blazing-fast CSV reader in Rust*. Retrieved from Medium: https://medium.com/@speedcraft21/zero-copy-or-bust-designing-a-blazing-fast-csv-reader-in-rust-511ae77237bd

MIT. (n.d.). *Using threads to run code simultaneously*. Retrieved from MIT: https://web.mit.edu/rust-lang_v1.25/arch/amd64_ubuntu1404/share/doc/rust/html/book/second-edition/ch16-01-threads.html

Moya, F. (2025, February 25). *Async Rust: When to use it and when to avoid it*. Retrieved from WyeWorks: https://www.wyeworks.com/blog/2025/02/25/async-rust-when-to-use-it-when-to-avoid-it/

Nahar, S. (2024, January 25). *Deadlock issues in Rust's DashMap: A practical case study*. Retrieved from Medium: https://savannahar68.medium.com/deadlock-issues-in-rusts-dashmap-a-practical-case-study-ad08f10c2849

Nkurikiyeyezu, K. (2022, November 2). *Fundamentals Of Real-time Systems.* Retrieved from https://qiriro.com/ecs6264/static_files/lectures/L2/handouts/2.1-rt-fundamentals.pdf

O'Sullivan, M., Voessner, S., & Wegener, J. (1998). Testing temporal correctness of real-time systems-a new approach using genetic algorithms and cluster analysis. *EuroSTAR'98*, 1-20.

Phelps, J. (2019, February 1). *Backpressure explained — the resisted flow of data through software*. Retrieved from Medium: https://medium.com/@jayphelps/backpressure-explained-the-flow-of-data-through-software-2350b3e77ce7

Programiz. (n.d.). *Rust stack and heap*. Retrieved from Programiz: https://www.programiz.com/rust/stack-and-heap

Serde. (2022). *Understanding deserializer lifetimes*. Retrieved from Serde: https://serde.rs/lifetimes.html

Stuart, D. (2023, September 25). *Rust: The joy of safe zero-copy parsers*. Retrieved from Medium: https://itnext.io/rust-the-joy-of-safe-zero-copy-parsers-8c8581db8ab2

Vallabhaneni, V. (2025). C++ memory safety and integrity evaluation. *Research Archive of Rising Scholars*, 1-7.

Vasdeva, R. (2023, December 14). *Applications, advantages, and disadvantages of stack*. Retrieved from Medium: https://medium.com/@raghav.vasdeva/applications-advantages-and-disadvantages-of-stack-6ede5f8af5f0

Vidyalaya. (2026, March 30). *Multilevel queue scheduling (MLQ) algorithm question with step-by-step solution + PYQs*. Retrieved from CS Vidyalaya: https://www.csvidyalaya.com/Operating-system/scheduling-algorithms/multilevel-queue-scheduling.html

Wang, Y. (2017). Research on the execution time analysis technology of the worst case system in real time system. *2017 9th International Conference on Measuring Technology and Mechatronics Automation (ICMTMA)*, (pp. 397-402). Changsha, China.

Watari, S. (2009). Space weather and real-time monitoring. *Data Science Journal, 8*, S78-S84.

Xu, H., Chen, Z., Sun, M., Zhou, Y., & Lyu, M. R. (2021). Memory-safety challenge considered solved? An in-depth study with all Rust CVEs. *ACM Transactions on Software Engineering and Methodology, 31*(1), 1-25.

Yang, Q. (2023). Future directions and potential advancements in using C language in machine learning and artificial intelligence.